

(iii) the Routing Transformer [19]. Reformer [18] changes the dense attention layer twofold: (i) it tightens vector representations of queries and keys, (ii) it sets their norm to be equal to 1. Reformer is the first paper to propose LSH for clustering queries and keys. In Reformer, instead of using Asymmetric LSH, the authors use Angular distance LSH for clustering. This works because of (i), (ii), i.e. the Maximum Inner Product Search problem is equivalent to the Nearest Neighbor Search problem. We consider SMYRF as a generalized version of Reformer, since it employs Asymmetric LSH clustering to enable grouping of queries and keys that (i) do not have the same vectors, (ii) possibly live outside or inside the unitary d -dimensional disk. Apart from this, SMYRF and Reformer are similar: both networks sort vectors based on their LSH hash and both have linear attention complexity. Sinkhorn [20] proposes a differentiable sorting module for clustering queries and keys. The sorting layer is trained end-to-end with the rest of the model. It has only been shown to work well for training from scratch and not for fine-tuning of pre-trained models. Routing Transformer [19] proposes k -means clustering. In general, vectors that have small Euclidean distance are not guaranteed to have big inner product. To alleviate this, in Routing Transformer queries and keys are forced to have exactly the same vector representations and are also mapped to a d -dimensional unitary disk, exactly as Reformer proposed. Because of these changes, also this method cannot be applied to pre-trained models. Routing transformer has some other weaknesses as well: (i) the complexity is $O(N^{1.5})$ instead of $O(N \log N)$ which is the attention complexity of SMYRF and Reformer and (ii) the clusters are not guaranteed to be balanced. To solve (ii), [19] proposes to keep the top- k vectors in each cluster. However, this is not guaranteed to work well since it depends on the clusters ordering.

Comparing to the aforementioned methods, SMYRF is the only method that assigns dynamically queries and keys in clusters and can be applied to pre-trained models. Due to its portability, SMYRF is the first sparse attention model to report GLUE results on par with the underlying models. As we showed, SMYRF can be used interchangeably with dense attention before, during and after training. It also has linear attention complexity, similarly to Reformer. To the best of our knowledge, we are also the first to prove that the problem that all these methods are trying to solve is NP-hard.

The optimization problem that SMYRF tries to solve is connected to the problem of bi-clustering [58]. Indeed, as shown in the proof of Theorem 3, the goal in Attention Biclustering is to find a clustering of rows and columns of a matrix that maximizes the sum of the values of the clusters, where each value at position (i, j) depends on the inner product of query i and key j . For bi-clustering, iterative algorithms have been proposed [59]. Iterative techniques cannot be applied in the context of attention in which everything happens in a parallel fashion for fast execution in modern hardware.

Finally, there are a lot of others not attention related techniques that can be used to save memory and offer speedups. Examples of such techniques include knowledge distillation [60, 61], reversible layers [62], gradient checkpointing [63], quantization [64] and pruning [65, 66]. SMYRF and all these innovations are not mutually exclusive, i.e. they can be used together for maximum efficiency.

6 Conclusions

In this work we presented SMYRF, a novel type of balanced clustering to approximate attention. It is based on Asymmetric LSH with novel transformations and an adaptive clustering scheme. As it does not require changes to attention, SMYRF is the first sparse attention method that can be applied directly to pre-trained models. We showed powerful experimental results, in terms of performance, memory and speed. We also defined the underlying optimization problem that SMYRF tries to solve and we proved it is NP-hard. The strong experimental performance of SMYRF inclines us to believe that good approximation algorithms exist for this problem. Proving approximation guarantees for our method and discovery of better approximation algorithms are left for future work.

7 Acknowledgements

We would like to wholeheartedly thank the TensorFlow Research Cloud (TFRC) program that gave us access to v3-8 Cloud TPUs and GCP credits that we used to run our Computer Vision experiments. This research has been supported by NSF Grants CCF 1763702, 1934932, AF 1901292, 2008710, 2019844 research gifts by Western Digital, WNCG IAP, computing resources from TACC and the Archie Straiton Fellowship.

8 Broader Impact

Our main contribution is to reduce the computational requirements for machine learning models with attention-layers. Thus, any broader impact is likely to come from making these models more efficient in both memory impact and inference speed. We expect that this will be mostly a good thing since it democratizes the use of big attention layers: those who want to use such models but for whom the computational resources required are too great (like university labs) will now have an easier time. Moreover, GANs and language models will become easier to deploy on phones or other embedded devices. Further, more efficient training reduces the environmental and energy footprint of deep learning research. As the number of parameters of Transformer models grows, the latter becomes critical [67].

Negative consequences are also possible: The idea of DeepFakes [68] has been well-discussed elsewhere; a technique that makes these easier to create clearly has downsides. On the other hand, any sufficiently determined actor (e.g. a nation-state attempting to commit election-fraud) already has access to such technology, so perhaps the marginal negative impact will not be that large. Still, whenever computational requirements are reduced, the ease of taking bad actions increases along with the ease of taking good actions.

Finally, the technique proposed in this paper relies heavily on the assumption that attention maps are approximately sparse. It's possible (though we have no particular reason to think that this has happened or would happen) that, at some intermediate layer of a complicated neural network, enforcing sparsity when the ground-truth maps are non-sparse could result in ignoring salient features of atypical data points, thus resulting in fairness-related issues. Determining whether these approximations cause fairness issues in general could be an interesting subject for future work.

Supplementary Material

SMYRF:

Efficient Attention using Asymmetric Clustering

Giannis Daras
 Computer Science Department
 The University of Texas at Austin
 giannisdaras@utexas.edu

Augustus Odena
 Google Research
 augustusodena@google.com

Nikita Kitaev
 Google Research
 kitaev@cs.berkeley.edu

Alexandros G. Dimakis
 ECE Department
 The University of Texas at Austin
 dimakis@austin.utexas.edu

9 NP-hardness of Attention Biclustering

To prove Theorem 1, we first prove the following lemma.

Lemma 1. *The optimization problem:*

$$\min_{\mathcal{C}_t^L \in \mathcal{C}^L} \|\hat{P}_\epsilon - P\|_F^2$$

is NP-hard.

Proof of Lemma 1. We will show that this problem is NP-hard, by showing that if we could solve in polynomial time all instances of this problem, we could solve in polynomial time the 3-dimensional matching problem (3-DM), which is known to be NP-complete. Following the notation of the main paper, we define $\epsilon = e^{-a}$ and $\hat{P}_\epsilon$ denotes the queries-keys product matrix with $-a$ in positions that correspond to queries and keys that do not belong to the same cluster.

It holds that:

$$\begin{aligned} \min_{\mathcal{C}_t^L} \|\hat{P}_\epsilon - P\|_F^2 &= \min_{\mathcal{C}_t^L \in \mathcal{C}^L} \sum_{(q,k) \notin \mathcal{C}_t^L} (q \cdot k - (-a))^2 \\ &= \min_{\mathcal{C}_t^L \in \mathcal{C}^L} \left[\sum_{(q,k) \in \mathcal{Q} \times \mathcal{K}} (q \cdot k + a)^2 - \sum_{(q,k) \in \mathcal{C}_t^L} (q \cdot k + a)^2 \right] = \min_{\mathcal{C}_t^L \in \mathcal{C}^L} \left[- \sum_{(q,k) \in \mathcal{C}_t^L} (q \cdot k + a)^2 \right] \\ &= \max_{\mathcal{C}_t^L \in \mathcal{C}^L} \sum_{(q,k) \in \mathcal{C}_t^L} (q \cdot k + a)^2 \end{aligned} \tag{6}$$

Since for all given sets $\mathcal{Q}, \mathcal{K}$ we can create (in polynomial time) sets $\mathcal{Q}', \mathcal{K}'$ such that: $(q \cdot k + a)^2 = q' \cdot k', \quad \forall (q, k) \in \mathcal{Q} \times \mathcal{K}, (q', k') \in \mathcal{Q}' \times \mathcal{K}'$, the problem is equally hard to solving:

$$\max_{\mathcal{C}_t^L \in \mathcal{C}^L} \sum_{(q',k') \in \mathcal{C}_t^L} q' \cdot k' \tag{7}$$

We can refer to this latest optimization problem as the max-mass problem.

Now consider the case where: $|\mathcal{Q}| = |\mathcal{K}|$, $L = |\mathcal{Q}|/2$, i.e. for this problem instance we have the same amount of queries and keys and we want to group them optimally to clusters with the constraint that each cluster should contain exactly 2 queries and 2 keys.

Note that for 1 query and one key per cluster this becomes weighted bipartite matching (which is efficiently solvable). For 1 query and m keys per cluster this is a generalized matching problem, which is also polynomially solvable [69].

If we are able to solve the latter with a polynomial algorithm, then we can show that we can solve the 3-DM problem with a polynomial algorithm.

Any instance of the 3-DM problem can be expressed with finite, disjoint sets X, Y, Z and a set T of triples $(x, y, z) : x \in X, y \in Y, z \in Z$. Visually, we can depict any instance of a 3-DM as a graph with three disjoint vertex sets, with T containing the edges of the graph. For example, the 3-DM instance $X = \{1_{\text{red}}, 2_{\text{red}}\}, Y = \{1_{\text{blue}}, 2_{\text{blue}}\}, Z = \{1_{\text{green}}, 2_{\text{green}}\}, T = \{(1_{\text{red}}, 1_{\text{blue}}, 1_{\text{green}}), (1_{\text{red}}, 2_{\text{blue}}, 2_{\text{green}}), (2_{\text{red}}, 1_{\text{blue}}, 1_{\text{green}})\}$ is shown in (1,1) of Figure 3. We are looking for a set $T' \subseteq T$ in which every vertex is covered exactly once. Finding this solution, in case it exists, it is known to be an NP-hard problem. For this example, there is a valid solution, which is shown in (1, 2) of Figure 3.

We can transform any instance in the following way: we create one query and one key vector for each vertex $x \in X$ with the property that their inner product is some large positive constant $r_1 \in \mathbb{R}^+$. We can visualize this using red edges, following the previous example where we denoted with red color the vertices of X . We also set the inner product of any key vector that corresponds to vertex of X with all the other query vectors to be 0. Visually, a “missing” edge means that the inner product of the corresponding vectors is 0 (no-reward). We also create a key vector for each vertex $y \in Y$ with the property that if $(x, y, z) \in T$ for some z , then the key vector for y and the query vector for x have inner product r_1 , else 0. We can show the non-zero edges of this category visually with blue color, following the previous example. Note that blue and red edges are equivalent in terms of the inner product between the vertices they connect, since both have inner product r_1 . Finally, we create a query vector for each $z \in Z$ with the property that if $(x, y, z) \in T$ for some x then the key vector for y and the query vector for z have inner product r_2 , else 0 where $r_2 \in \mathbb{R}^+$ is a small positive constant. Again, we can show the non-zero edges of this category with green color, following the previous example. For the given example, the transformation is shown in (2, 1) of Figure 3.

We have hypothesized that we have a polynomial algorithm to solve the max-mass problem of (7). The key observation for our proof is that, by construction, the best cluster in terms of potential accumulated mass is a cluster with one red, one blue and one green edge, as the ones shown right of the dashed bar of Figure 4. Indeed, the only way to obtain a cluster of more mass is to group two blue vertices with two red vertices, as shown in (1, 1) of Figure 4. By doing that, you earn one more r_1 compared to the clustering shown in (1, 2) of Figure 4, but you lose $2 \cdot r_1$, which are the rewards that they red keys could give (as they are left with no connections). Thus, the two clusterings on the right side of Figure 4 are preferable compared to any other potential two clustering that can be obtained by choosing the left grouping.

Since we have proved that the best possible clustering is one with one red, one blue and one green edge, it is now left to prove that if there is a 3-DM, then it is possible to group all queries and keys into clusters with this optimality property. Indeed, if there is a 3-DM, we can cover each vertex exactly one time, by matching any vertex of X with a vertex from Y and a vertex from Z . With our transformation, this means that we can group each red node with itself and one blue and one green vertex, which is an optimal cluster as it contains one red, one blue and one green edge. Thus, solving polynomially our problem would mean that we could also solve in polynomial time the 3-DM, which is known to be NP-hard.

□

Proof of Theorem 1. We will show that if we can solve in polynomial time the problem: $\min_{\mathcal{C}^L} \|\sigma(\hat{P}_0) - \sigma(P)\|_F^2$, then we can also solve in polynomial time the problem $\min_{\mathcal{C}^L} \|\hat{P}_\epsilon - P\|_F^2$ (for an appropriate ϵ) which we have proven to be NP-hard.